Nanyang Technological University

SC2002 Object-Oriented Design & Programming

Restaurant Rush

Design and development report

November 2026

Runtime	JDK 11 or later; offline command-line game
Repository	https://github.com/SeraphinaXYX/NTU-SC2002-Restaurant-Rush
Stage tags	`stage-1`, `stage-2`, `stage-3` (published)

Group members

Name	Matric No.	Lab Group	Contribution
Liang Rui	U2520207B	SCEF	Stage 1 core, Stage 1 class diagram, design section
Xie Yixuan	U2523645J	SCEF	Stage 2 VIP/Critic, Stage 2 class diagram, reflections
Angel Chua	U2522326K	SCEF	Stage 3 features, Stage 3 class diagram, evolution section
Zhu Yuean	U2421119C	SCEF	Tests, sequence diagram, functional evidence

Contents

1 UML Class Diagrams **2**
1.1 Stage 1 class diagram . . . 2
1.2 Stage 2 class diagram . . . 4
1.3 Stage 3 class diagrams (detailed views) . . . 6

2 UML Sequence Diagrams **12**
2.1 How the choice changes the outcome . . . 16

3 Design consideration **16**
3.1 Responsibilities and cohesion . . . 16
3.2 Principles reflected in this code . . . 16
3.3 Encapsulation, composition and alternatives . . . 17

4 Stages and development evidence **17**
4.1 Source identities and tag status . . . 17
4.2 Stage 1 to Stage 2 . . . 18
4.3 Stage 2 to Stage 3 . . . 18

5 Functional evidence **18**
5.1 Independent rule checks . . . 19
5.2 How a manager decision changes the outcome . . . 19
5.3 Coverage limits . . . 19

6 Reflections **19**
6.1 Technical reflection . . . 19
6.2 Personal reflections . . . 20

7 Video **21**

1 UML Class Diagrams

We show key fields and operations and omit routine getters and constructors. `+`, `-` and `#` mean public, private and protected. Underlining marks static members. Fields marked `final` cannot be reassigned. A hollow triangle denotes inheritance, a filled diamond composition, a hollow diamond aggregation, an arrow association, and a dashed arrow dependency. Multiplicities describe one running game. State holders are classes of public static final String constants, not Java enums.

1.1 Stage 1 class diagram

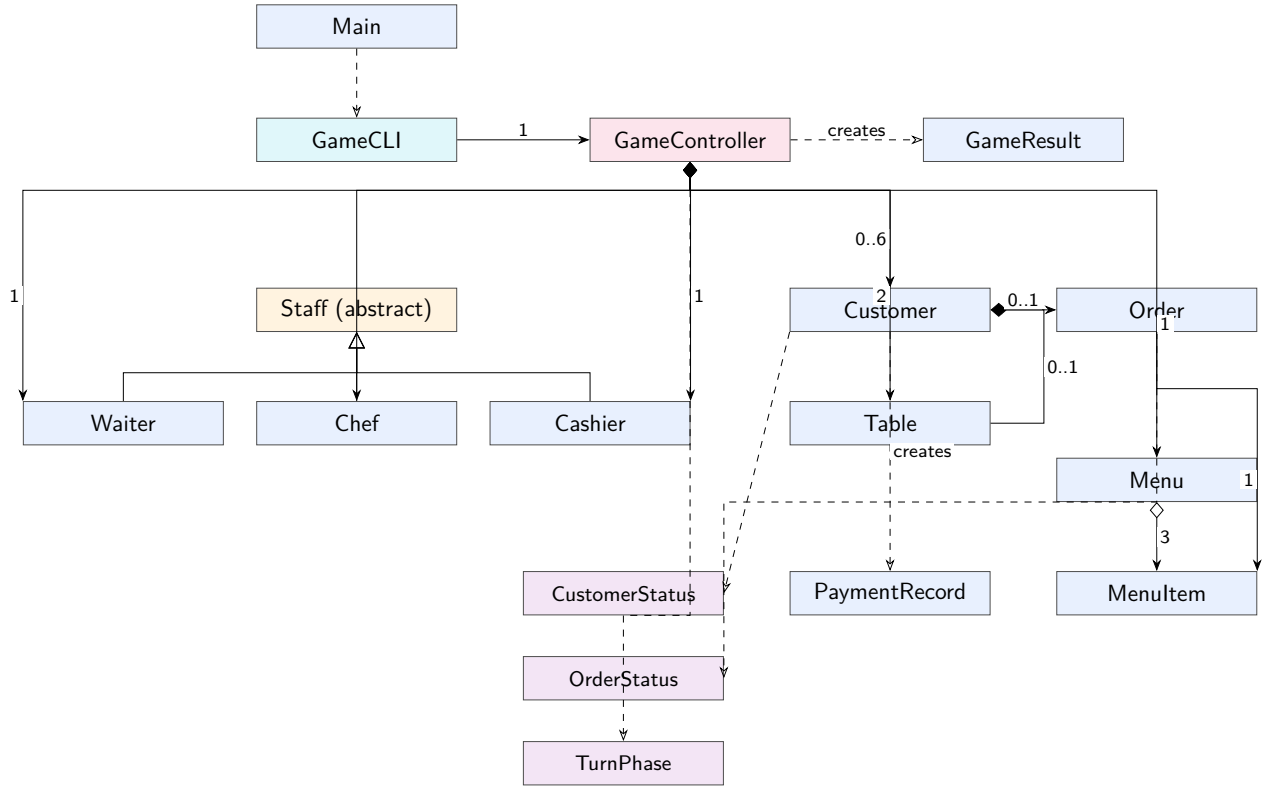


Figure 1: Stage 1 class diagram. Three concrete staff roles inherit the one-task availability from abstract Staff. Customer owns at most one Order; GameController owns all domain objects. The three String-constant holders act as state enumerations. 17 classes total.

1.2 Stage 2 class diagram

Stage 2 adds polymorphic customer types and a schedule that constructs them. Three new classes (shaded darker) extend the Stage 1 design without changing existing interfaces.

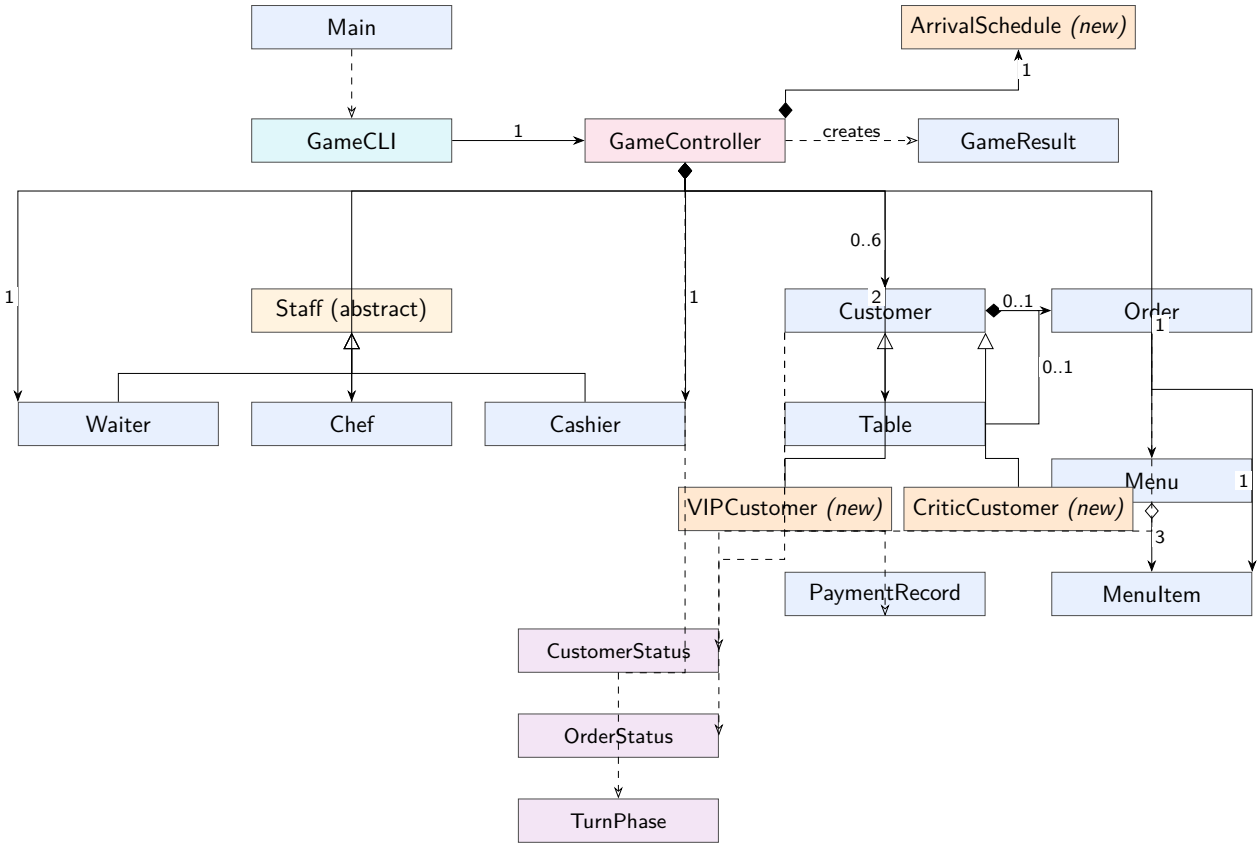


Figure 2: Stage 2 class diagram. Shaded boxes are new. ArrivalSchedule constructs VIP and Critic subclasses via a factory method. VIPCustomer overrides patience loss and adds a 10% bill discount; CriticCustomer overrides meal selection to prefer the most expensive well-prepared dish. Existing Customer references dispatch the differing rules via polymorphism. 20 classes total (+3 from Stage 1).

1.3 Stage 3 class diagrams (detailed views)

The following complementary views describe the final Stage 3 implementation. They share class names across views; repeated boxes refer to the same classes. Figure 7 gathers the ownership, associations and state dependencies.

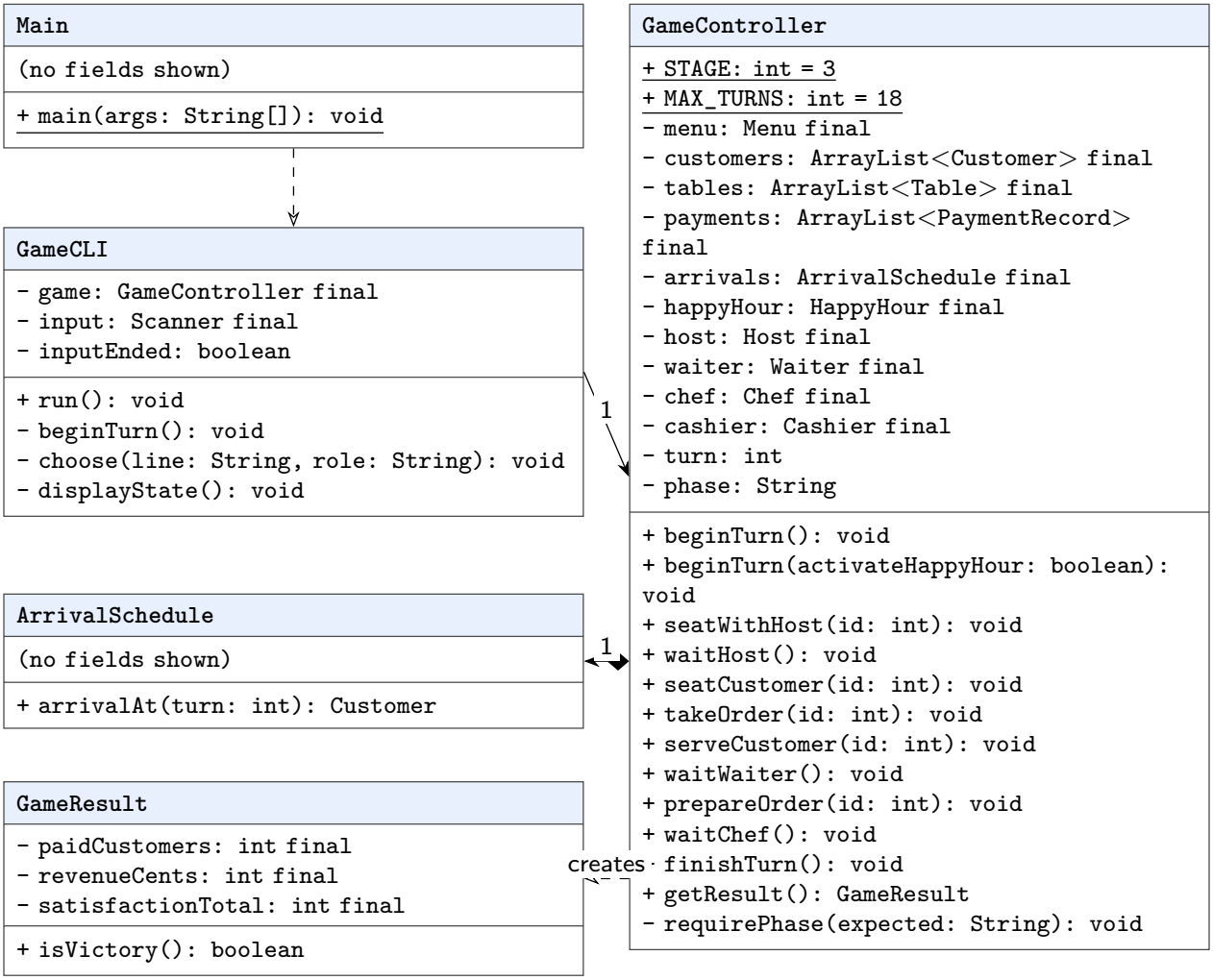


Figure 3: Boundary and turn control. Main constructs the CLI and controller; GameCLI requests actions and catches invalid choices. GameResult checks all three thresholds using unrounded payment totals.

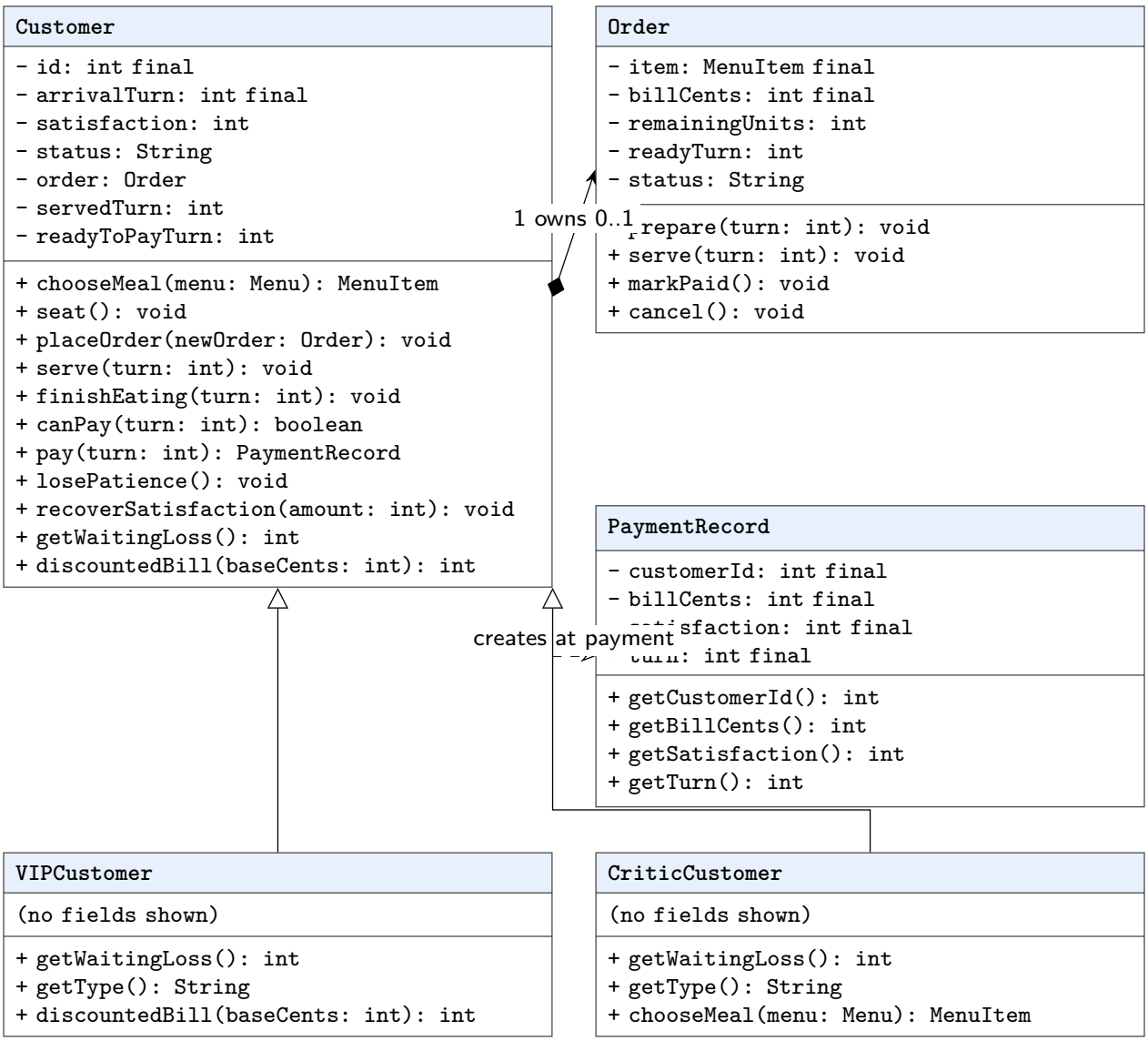


Figure 4: Customer and order lifecycle. Encapsulation: transition methods guard private state and timestamps. Composition: each Customer owns at most one Order. Polymorphism: Customer is the concrete Regular type; VIP and Critic override selected behaviour through Customer references. PaymentRecord preserves the bill and satisfaction at payment.

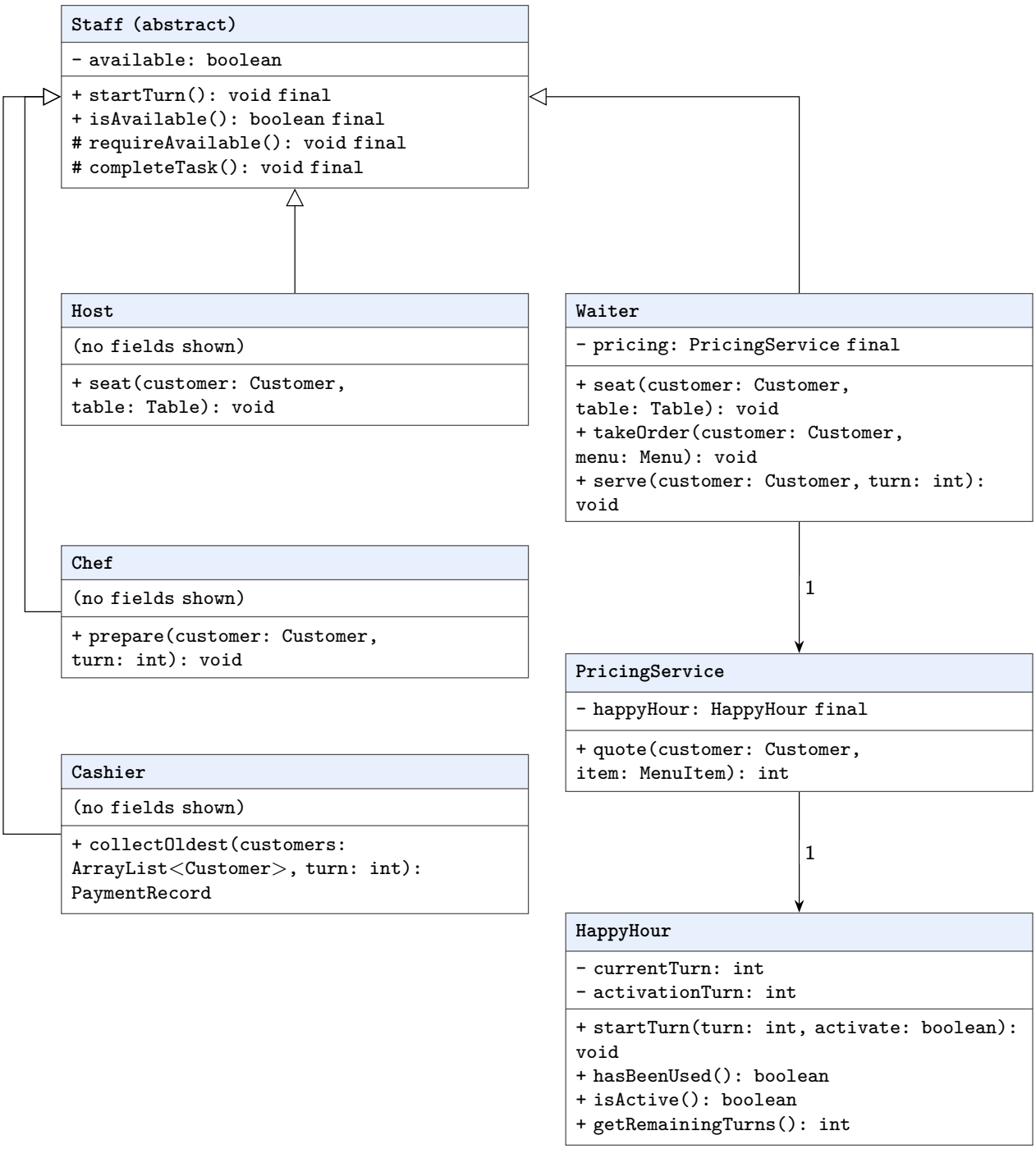


Figure 5: Staff and pricing. Staff shares one-task availability without declaring an unrelated generic work operation. Host and Waiter depend on Table; Chef depends on Order; Cashier depends on Customer and creates no payment until a customer is eligible. PricingService references the same HappyHour owned by the controller.

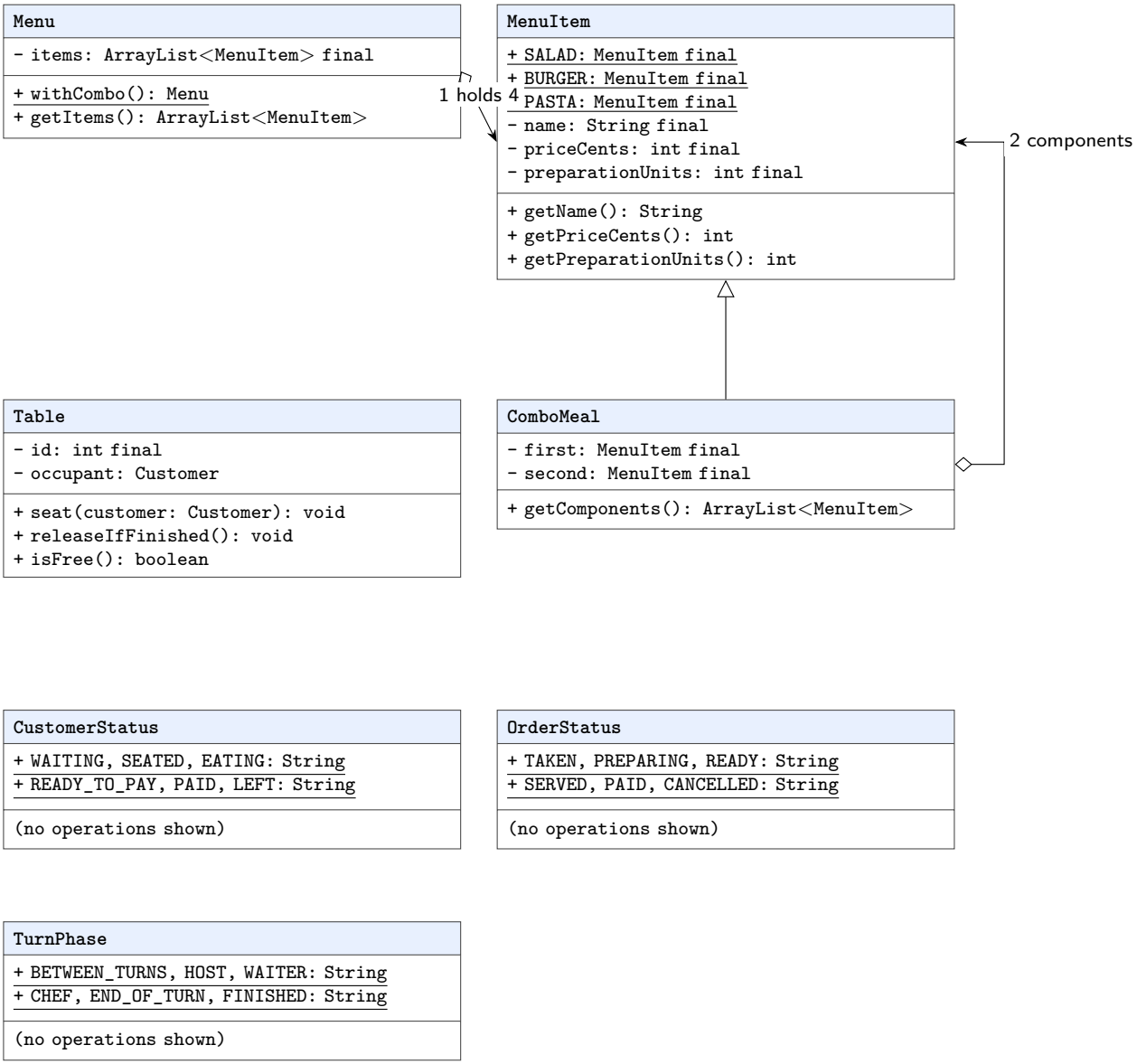


Figure 6: Menu, table and String constant holders. Menu items and combo components are shared immutable references, so hollow diamonds indicate aggregation. Each Table references 0..1 Customer; each Customer occupies 0..1 Table. An Order references exactly one MenuItem. Customer, Order and GameController depend on their respective status/phase constant classes. All constants shown are public static final; grouped names denote separate fields.

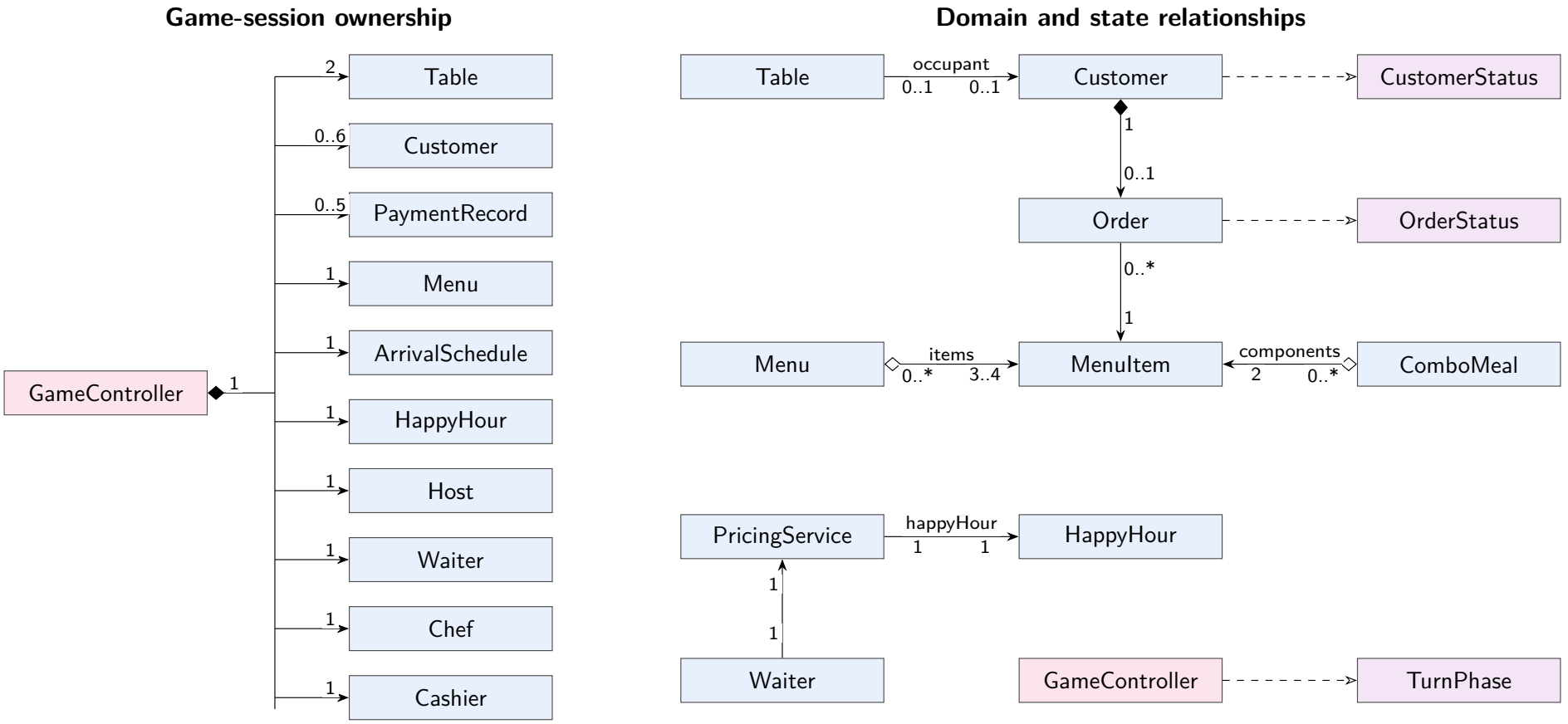


Figure 7: Relationship overview. The shared composition trunk on the left means that each listed object is owned by one GameController; numbers beside the destinations are per-game counts. On the right, endpoint numbers are association multiplicities. Menu and ComboMeal share immutable menu entries (hollow diamonds); Customer owns its Order (filled diamond). State dependencies point to String constant classes, not enums. Repeated names denote the same classes as in the detailed views.

Implementation notes. Menu returns a copy of its list, but the immutable menu entries are shared. GameController similarly returns container copies for customers, tables and payments. Customer, Order and Table enforce their own valid transitions; the CLI receives exceptions and retries the same role. The model deliberately uses ordinary classes, ArrayLists and loops rather than additional interfaces or a framework.

2 UML Sequence Diagrams

The scenario follows VIP C2, who arrives on turn 4 (with C1 at the first table and the second table free). The manager activates Happy Hour for turns 4–5 and asks the Host to seat C2, leaving the Waiter free to take an order in the same turn. Burger costs 1500 cents; VIP pricing gives 1350 and Happy Hour gives a locked bill of 1080 cents. After ordering succeeds, the CLI announces the meal, preparation and locked bill before asking for the Chef's task. The Chef finishes it on turn 4, the Waiter serves on turn 5, eating finishes on turn 6, and payment is eligible on turn 7 if no earlier eligible customer takes the Cashier's task.

The three panels form one scenario and use real method names. Solid arrows are calls and dashed arrows are returns. Combined fragments carry guards; the horizontal dashed separator divides alternatives within one alt frame. Creation arrows meet a new object's header, and that object's lifeline starts there. Existing objects carry on across panels. Routine getters, availability resets and operations on other customers are omitted. The regenerated functional evidence checks the timing and payment values; these are deterministic choices, not seeded random draws.

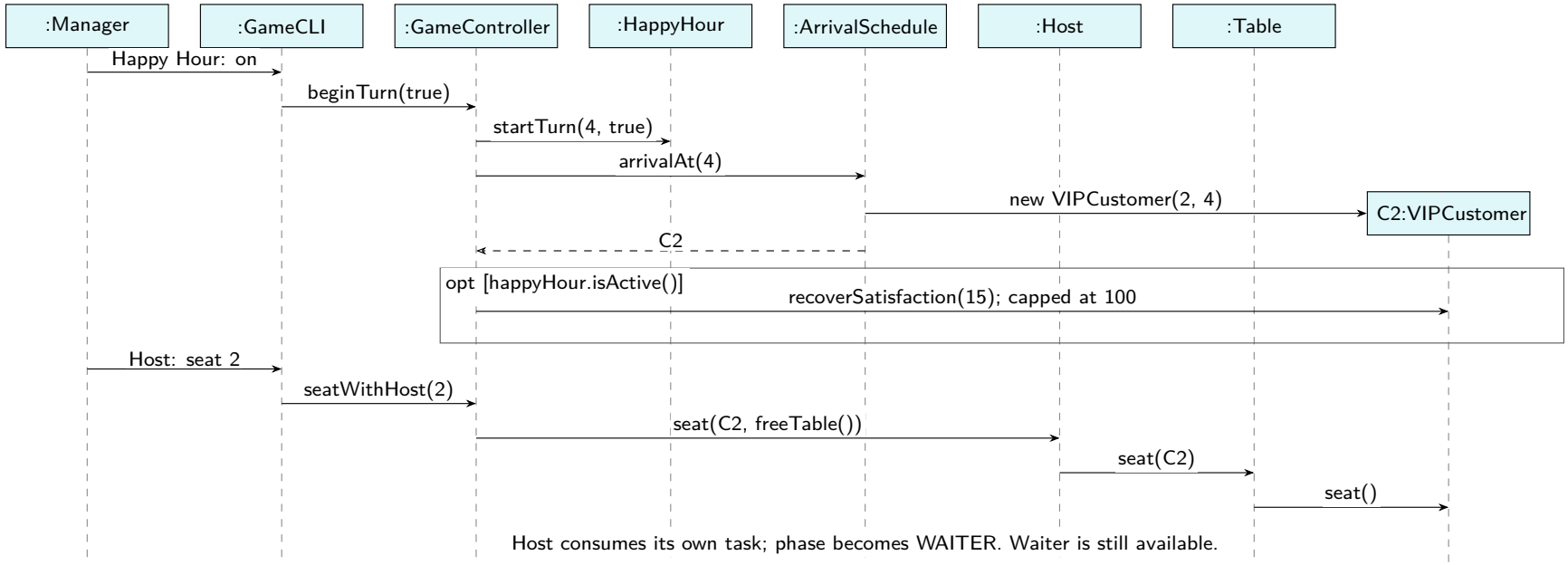


Figure 8: Turn 4 arrival and seating. A free table is available. The Host seats C2, permitting ordering during the Happy Hour window without using the Waiter task.

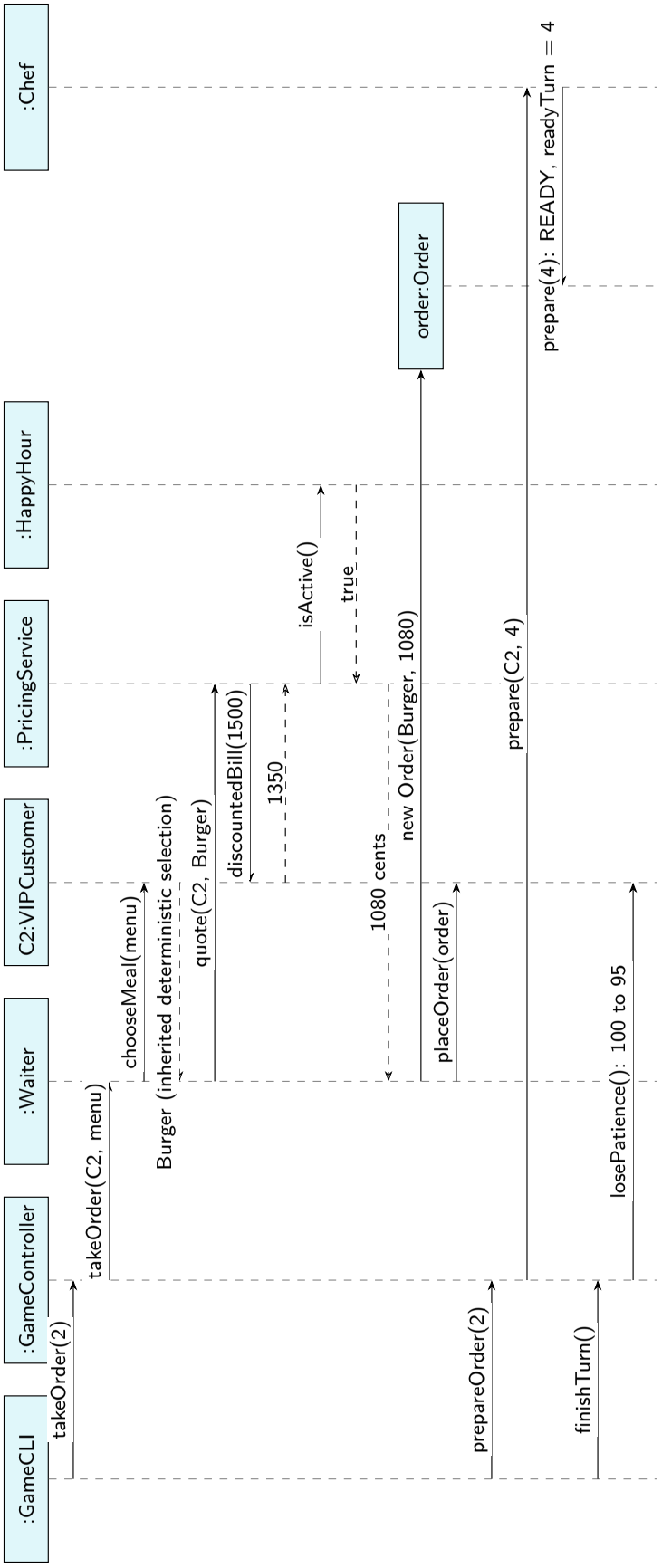


Figure 9: Turn 4 ordering and preparation. Pricing applies the VIP discount before Happy Hour. Order stores its final bill once. The Waiter phase has already ended when the Chef finishes, and Order also guards against service in its ready turn.

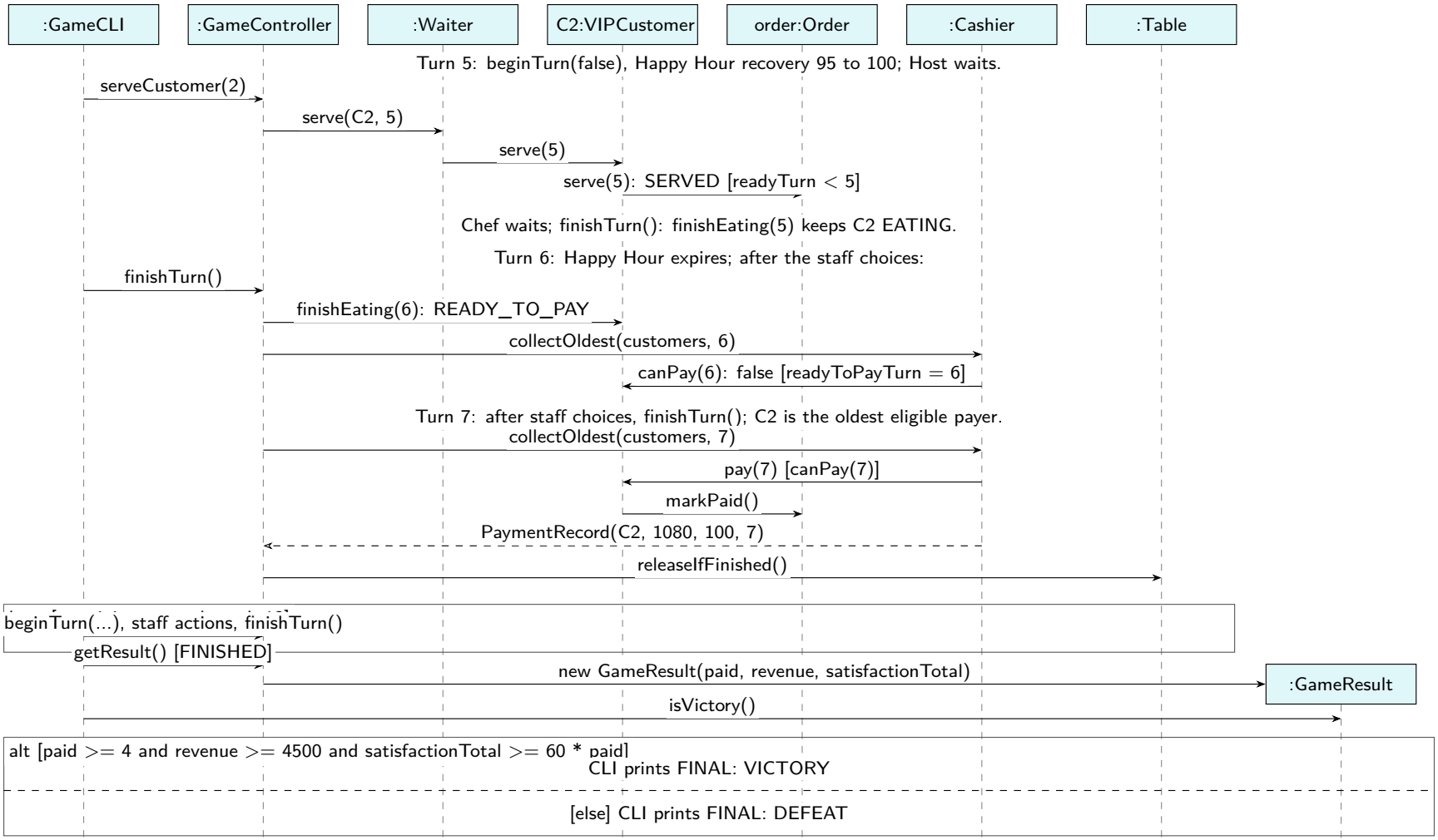


Figure 10: Turns 5–7 and the final result check. Eating is updated before payment selection; the strict timestamp guard still defers payment until turn 7. The immutable PaymentRecord is stored by the controller. The combined alternative tests the final game totals, not this one payment alone.

2.1 How the choice changes the outcome

Using the Host on turn 4 frees the Waiter's single task for ordering. If the manager instead makes the Host wait and the Waiter seat C2, ordering cannot happen until turn 5, and the earliest preparation, service and payment each move one turn later. Happy Hour activation trades revenue for patience: the same Burger costs 1080 cents during the window instead of the VIP-only 1350 cents. Recovery on turn 5 removes C2's five-point waiting loss from turn 4, so this payment records satisfaction 100 instead of 95 without Happy Hour. These are local consequences; the complete game's victory still depends on all payments.

Validation and retries. GameCLI repeats its role loop while the controller remains in that phase. If an action fails, an `IllegalArgumentException` or `IllegalStateException` is caught, the reason is printed and the phase is unchanged. Successful domain work precedes `completeTask()` and the controller's next-phase assignment. The successful scenario above omits these exceptional calls; no `ActionResult` class or `GameView` is used by this implementation.

3 Design consideration

3.1 Responsibilities and cohesion

Abstraction	Responsibility
`Main`, `GameCLI`	Construct the program, collect commands, explain invalid choices and display state. The CLI requests actions through the controller.
`GameController`	Coordinate turn phases, staff tasks, arrivals and aggregate results. It owns the game collections and has no console dependency.
`ArrivalSchedule`	Create the scheduled customer type without putting type-dependent patience/pricing rules in the turn loop.
`Customer`, `VIPCustomer`, `CriticCustomer`	Own service state and satisfaction; dispatch waiting loss, meal preference and customer discount. Concrete `Customer` represents a Regular.
`Order`	Own preparation progress, readiness timestamps, locked bill and legal food-state transitions.
`Table`	Hold one occupant until payment or departure; centralise seating.
`Staff` and its four roles	Share the one-task allowance while each subclass exposes its own meaningful operations.
`Menu`, `MenuItem`, `ComboMeal`	Describe immutable item prices/preparation and compose the two existing combo components.
`HappyHour`, `PricingService`	Track activation/duration and calculate the order-time quote with the required discount order.
`PaymentRecord`, `GameResult`	Preserve completed-payment facts and evaluate the exact victory thresholds.
`TurnPhase`, `CustomerStatus`, `OrderStatus`	Hold named String constants. These are not Java enums.

3.2 Principles reflected in this code

SRP. The CLI handles interaction, the controller schedules the turn, Order guards preparation/serving/payment transitions, and PricingService quotes the bill. These responsibilities can

be discussed and tested separately. The small forecast shown by GameCLI is presentation logic with timing assumptions; keeping it beside the display is a simplicity trade-off.

OCP. Customer references dispatch `getWaitingLoss()`, `chooseMeal()` and `discountedBill()` to the applicable subclass. Adding VIP/Critic behaviour did require setup and a pricing hook in Stage 2; the claim is reduced coupling, not zero changes to existing code. Adding Host and the pricing policy did not require rewriting Chef or Cashier.

LSP. Regular, VIP and Critic customers follow the same seat/order/serve/ eat/pay flow. Their overrides change documented preferences, patience and pricing while preserving that service contract. ComboMeal supports the same MenuItem getters used by ordering and cooking.

ISP. There is no interface hierarchy in this small implementation. Staff shares only task availability; it does not force Cashier to implement cooking or Chef to implement seating. This is a focused API choice, not a claim that an unused interface pattern has been implemented.

DIP. Waiter receives a PricingService through its constructor and GameCLI receives its controller and Scanner. These explicit dependencies aid construction and testing. Both still depend on concrete classes; full interface-based dependency inversion is deliberately not claimed.

3.3 Encapsulation, composition and alternatives

Private fields and domain operations guard state. A failed action leaves the controller in the same phase, and Staff completes a task only after a valid action. Order and Customer timestamps prevent same-turn serving and early payment, respectively. PaymentRecord is immutable, so later events cannot change recorded revenue or paid satisfaction. Returned ArrayLists are container copies containing the same domain objects; this is a deliberately limited boundary for a six-customer CLI.

Customer has at most one Order. GameController owns its staff, tables and payment records. ComboMeal holds references to two existing immutable MenuItem components; this is aggregation of shared menu objects rather than exclusive ownership of them. The waiting queue is derived from customer state in arrival order, avoiding a second membership collection that could become inconsistent.

An alternative was one large controller with customer-type conditionals, parallel order arrays and discount branches. It would reduce the number of files, but entangle service transitions, pricing and display. The chosen model keeps those rules in the objects that own the affected state. A more generic engine with interfaces for every phase was also unnecessary for 18 turns and six customers. Named String constants are familiar to the course, though enums would give stronger compile-time state checking in a larger project.

4 Stages and development evidence

4.1 Source identities and tag status

Stage	Tag	Evidence
Stage 1	`stage-1`	Regular customers and complete base game; independent compile, victory and defeat.
Stage 2	`stage-2`	VIP and Critic extensions; independent compile, victory and defeat.
Stage 3	`stage-3`	Host, combo, Happy Hour and order announcements; independent compile, victory and defeat.

All three annotated tags are published on the remote repository. Each tagged commit compiles and runs independently.

4.2 Stage 1 to Stage 2

Compare Stage 1 and Stage 2.

- New: `ArrivalSchedule.java`, `VIPCustomer.java` and `CriticCustomer.java`.
- Changed: `GameController.java` delegates arrival construction; `Customer.java` adds the discount hook; `Waiter.java` calls it.
- Changed: `README.md` explains the new behaviour. No files were deleted.

These are both registration and behavioural changes: setup selects the subtype, then the existing Customer references dispatch the differing rules.

4.3 Stage 2 to Stage 3

Compare Stage 2 with Stage 3.

- New domain files: `Host.java`, `ComboMeal.java`, `HappyHour.java` and `PricingService.java`.
- Changed: `GameController.java` and `TurnPhase.java` add the Host phase and turn-start policy; `Waiter.java` requests a quote.
- Changed: `Menu.java` registers the combo; `Customer.java` adds recovery and a served timestamp getter; `Order.java` adds a ready timestamp getter.
- Changed: `GameCLI.java` collects Stage 3 choices, displays policy duration and explains earliest timing.

Historical source, rather than a Stage 3 feature switch, establishes the real earlier stages. Each tagged revision can be checked out and compiled independently.

5 Functional evidence

The integration test runner compiles each historical snapshot independently with `javac --release 11 -Xlint:all`, then compiles the current source. The recorded run uses Corretto OpenJDK 11.0.29. All ten CLI runs and 43 independent rule checks passed. Current source is PR #1's `b62d196` plus the successful-order announcement. Its reviewed source digest is pinned in the runner; current receipts identify the source commit or uncommitted snapshot and base revision.

Run `python3 tests/run_checks.py` from the repository root. The game itself needs only Java; Git and Python 3 are test tools. Temporary sources and class files are removed. Ordinary verification compares saved transcripts and does not overwrite them. Explicit regeneration still requires semantic checks to pass. Full source/input/output hashes and observed metrics are recorded in `tests/evidence/results.json`.

Fixture	Paid	Revenue	Paid sat.	Turns
`stage1-victory`	5	$66.00	79.2	18
`stage1-defeat`	0	$0.00	N/A	18
`stage2-victory`	5	$64.50	78.0	18
`stage2-defeat`	0	$0.00	N/A	18
`stage3-victory`	5	$77.40	81.6	18
`stage3-defeat`	0	$0.00	N/A	18
`head-victory`	5	$77.40	81.6	18
`head-defeat`	0	$0.00	N/A	18
`head-invalid`	5	$77.40	81.6	18
`head-no-happy-hour`	5	$80.10	80.6	18

Table 2: Freshly generated functional results; zero-paid runs are defeats, the others victories.

The Stage 1 and 2 fixtures use their exact historical commits; Stage 3 uses `bae23ff`; head fixtures use the reviewed working tree based on `b62d196`. All victories serve five customers with no unhappy departures. All wait-only defeats have two departures by closing; later arrivals have not all exhausted patience. Stage 2 includes the VIP and Critic. The head-invalid run inserts four rejected choices and still matches the valid victory metrics, checking that invalid input does not consume the staff task or silently advance play. Current-source runs also assert the exact new-order announcement and that it precedes the same-turn Chef prompt, including the VIP and Happy Hour bill. This regression failed on the old source and passed after the CLI patch. Historical stage transcripts and all outcome metrics remain unchanged.

5.1 Independent rule checks

The Java harness checks the Pasta worked example separately from the arrival schedule: seat on turn 1; order and first preparation unit on turn 2; READY on turn 3; serve on turn 4; READY_TO_PAY on turn 5; pay $18.00 on turn 6. Satisfaction is 92, 84 and 76 after the three unserved turns, then remains 76. Same-turn serving/payment are rejected and the table is released after payment.

Other assertions check the $24.30 three-unit combo, VIP Burger with Happy Hour at $10.80, the two-turn/once-only policy, recovery cap, paid/departed immunity, price locking, polymorphic waiting losses, abandonment cancellation, staff capacity, unpaid revenue exclusion and exact victory thresholds. Transcript comparison alone is not the basis for these claims.

5.2 How a manager decision changes the outcome

The head-no-happy-hour fixture uses exactly the same staff commands as the head-victory fixture, but leaves Happy Hour off. Each customer's food is identical because this implementation's choices are deterministic by ID and menu. With Happy Hour on turn 4 the final result is $77.40 and satisfaction 81.6; without it the result is $80.10 and 80.6. Both pay five customers and win. The decision costs $2.70 for one point of final average satisfaction. It is a measured trade-off, not a claimed victory reversal or optimal strategy. The sequence diagram traces the VIP whose bill changes from $13.50 to $10.80 and whose recorded satisfaction changes from 95 to 100.

5.3 Coverage limits

The focused suite covers ten scripted CLI journeys, all stage compiles and the listed rules. It does not exhaust every manager strategy, certify remote tags, produce a recorded video or establish instructor acceptance. The exact receipts and inputs accompany the project, so outcomes can be reproduced and inspected.

6 Reflections

6.1 Technical reflection

Separating the controller from console input makes rules executable without simulating the terminal. Timestamp checks are particularly important: the visually similar READY, SERVED and READY_TO_PAY states must still occur on different turns. The independent rule checks complement complete CLI runs, which could otherwise repeat an error in both a transcript and its expected output.

The chosen scope keeps the existing small design and deterministic choices. It does not add a GUI, a generic engine or a new advice system. Those features are not required to explain the OO design and could increase the verification burden. Current limitations include concrete dependencies, String-based states and container copies that still expose domain-object references. These are explicit trade-offs rather than claims of a fully general production model.

6.2 Personal reflections

Liang Rui

My contribution covered development and verification of the Stage 1 baseline, its class diagram and the report's design discussion. This linked implementation, UML and design reasoning. Xie Yixuan contributed Stage 2 extensions, Angel Chua Stage 3 features, and Zhu Yuean automated tests and functional evidence.

The project strengthened my understanding of encapsulation and the single responsibility principle. `Order` guards its private state through valid operations, including rejecting service in the turn a dish becomes ready. This showed me how encapsulation protects game rules. Separating interaction in `GameCLI`, turn coordination in `GameController`, and rules in domain objects also made the many mechanisms easier to understand and verify.

My main challenge was systematically checking that these mechanisms worked together. Preparation, serving, eating, payment, satisfaction and table occupancy depend on one another, so I needed to consider both individual rules and their execution order.

During final preparation, I arranged AI-assisted verification using existing tests and targeted checks. All ten scripted gameplay outcomes and 43 existing domain assertions passed. A controlled Pasta scenario confirmed READY on turn 3, service on turn 4, readiness to pay on turn 5, and payment on turn 6: an $18.00 bill, satisfaction 76 and table release. Supplemental checks covered generated games, malformed input and deliberately faulty temporary copies. Codex executed the checks and wrote the additional harnesses under my direction; the original test suite remained Zhu Yuean's contribution.

AI accelerated development, but judgement and engineering decisions remained ours. The verification work reinforced that generated code must be understood and checked against requirements; plausibility alone does not establish correctness.

In a future project, I would agree on the baseline and responsibilities earlier, use shared checkpoints, and keep a common record of decisions and verification evidence. This would improve time management and reduce repeated work and administrative overhead, leaving more time for implementation and validation.

Xie Yixuan

I extended Stage 1 with the `VIPCustomer` and `CriticCustomer` subclasses, the `ArrivalSchedule` factory, and the Stage 2 class diagram.

The core concept I applied was the template-method pattern for discounts. Adding a `discountedBill(int baseCents)` hook to `Customer` — returning the value unchanged by default — let `VIPCustomer` override it with a 10% reduction without changing any call sites in `Waiter` or `GameController`. Keeping the override in the subclass, rather than adding a type check in the caller, is the polymorphism principle from lecture directly at work.

The trickiest part was `CriticCustomer.chooseMeal()`. The selection rule is: pick the item with the most preparation units, breaking ties by highest price. I initially wrote two separate loops, but that required two passes over the menu. Combining the condition into a single expression — `units > best.units || (units == best.units && price > best.price)` — let me handle both criteria in one pass and made the intent clearer.

Angel Chua

I built the Stage 3 features: the `Host` phase, `ComboMeal`, `HappyHour`, and `PricingService`.

The design decision I found most educational was using dependency injection for `PricingService`: instead of letting the service create its own `HappyHour` instance, it receives one from `GameController`. This keeps the service stateless and testable without a running game loop.

Modelling `ComboMeal` as a subclass of `MenuItem` rather than a separate concept also taught me composition within inheritance: the two component items are stored and exposed, but all callers that work with `MenuItem` references require no changes.

The hardest part was the integer-rounding rule. Using `(price * 90 + 50) / 100` for a 10% discount ensures half-cent values round up consistently. I initially used a floating-point cast, which produced rounding inconsistencies against the expected transcript, and switched to the integer formula after tracing the difference in a failing fixture run.

Zhu Yuean

My contribution spanned all three stages: I wrote the `.in`/`.out` test fixtures, the `RuleChecks` suite, the sequence diagram, and the `results.json` functional evidence.

Writing deterministic test fixtures taught me how much clarity a fixed input sequence provides. Each `.in` file is a complete transcript of player decisions; the matching `.out` file is the exact expected output. Running a stage tag against its fixtures gives an objective pass/fail signal that does not depend on manual inspection.

Drawing the sequence diagram challenged me to choose the right level of abstraction. I settled on the VIP + Happy Hour scenario starting at turn 4, because it shows the full discount chain: `Waiter.takeOrder()` calls `PricingService.quote()`, which calls `customer.discountedBill()` and then checks `happyHour.isActive()`. Showing both calls in sequence makes the layered discount policy visible in a way that the class diagram alone cannot.

7 Video

Segment	Required content	Presenter / Timestamp
1	Launch from the submitted project	Liang Rui, 0:00
2	Meaningful choices under limited staff capacity	Liang Rui, 2:05
3	READY → SERVED → payment	Xie Yixuan, 4:10
4	VIP or Critic behaviour	Xie Yixuan, 6:20
5	A Stage 3 feature (Host / ComboMeal / Happy Hour)	Angel Chua, 8:35
6	Reproducible victory and defeat	Angel Chua, 10:45
7	Stage comparisons and final diagrams	Zhu Yuean, 12:00